

1.1 Simple CNN

- Tamaños de los conjuntos de entrenamiento y validación descargados del *dataset* MNIST

	Alto de imagen	Ancho de imagen	N.º canales de imagen	N.º muestras
Entrenamiento	28	28	1	60000
Validación	28	28	1	10000

- Número de parámetros del modelo Simple CNN

	N.º parámetros entrenables
Simple CNN	813802

- Incluya las curvas de entrenamiento y validación para 10 épocas. Indique también la mejor precisión obtenida, y en qué época se logra este resultado

	Mejor precisión (validación)	Época con mejor precisión
Simple CNN	99.05%	Época 10

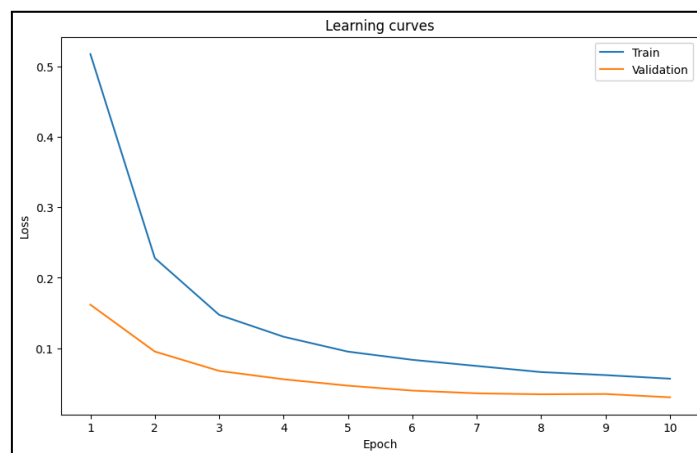


Figura 1: curvas de entrenamiento para la CNN de la práctica y MNIST.

- Comente las conclusiones sobre la evolución de la *loss* de entrenamiento y validación, con respecto a posibles problemas de sesgo (*high-bias*) o sobreajuste (*overfitting*). Indique si considera que continuar con más épocas de entrenamiento mejoraría el rendimiento del modelo.

El hecho de que la *loss* de entrenamiento se mantenga por encima de la de validación durante las 10 épocas es algo inusual. Lo normal es que la CNN tienda a obtener menor *loss* en entrenamiento (incluso con riesgo de sobreajuste), pues son precisamente los datos sobre los que optimiza, aprendiendo sus la distribución de los datos e incluso sobreajustándose a ella, aprendiendo ruido o particularidades de la

muestra en concreto. Sin embargo, en este caso, parece que el efecto del *dropout* está provocando que aun aprendiendo la red, su rendimiento en entrenamiento se vea limitado en este contexto, pues el dropout está activo durante la medición sesgando la métrica.

- Incluya la matriz de confusión obtenida. A partir de ella, indique cuáles son los dos casos de confusión entre clases que ocurren con mayor frecuencia y señale en la matriz los valores utilizados para identificarlos. Tras revisar los ejemplos de errores del notebook, incluya ejemplos de los casos de confusión identificados.

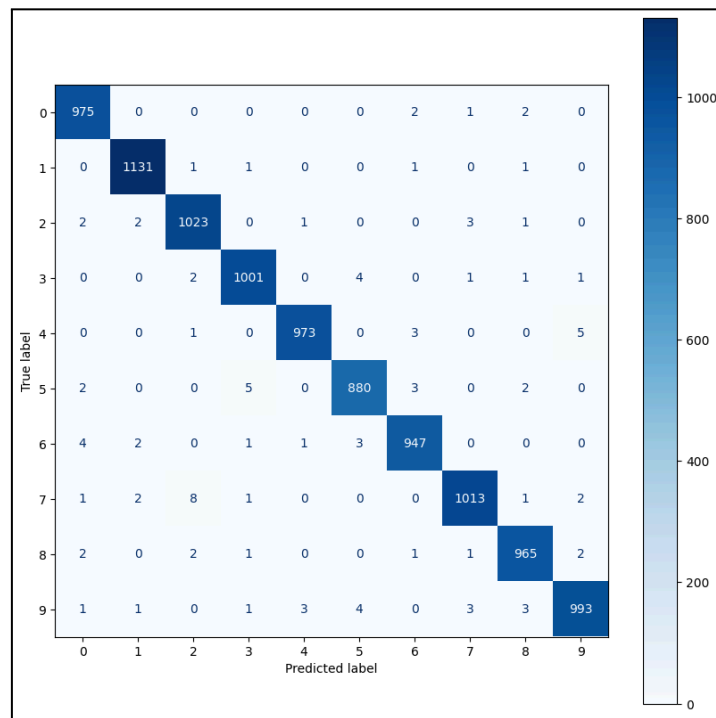


Figura 2: matriz de confusión para las clases reales y las predichas por el modelo.

La confusión que ocurre con mayor frecuencia es la predicción de dos para un dígito real de siete. Podemos ver, en la matriz de confusión, que esto ocurre 8 veces (eje x = 2, eje y = 7). También tiende a predecir dígitos nueve para cuatros reales (eje x = 9, eje y = 4) y dígitos tres con cinco reales (eje x = 3, eje y = 5); ambas con una frecuencia de 5. No obstante, habría que examinar las etiquetas, pues es posible que un humano haya etiquetado los dígitos erróneamente o bajo un supuesto que la CNN no está siendo capaz de interpretar (el humano ve algo parecido a la etiqueta que asigna pero que definitivamente no es un tal dígito).

- Comente las diferencias entre los gráficos t-SNE de las representaciones de las capas final e intermedia de la CNN, aplicadas a las imágenes del conjunto de validación. Para ello:
 - Señale en el gráfico de la capa intermedia qué muestras deberían corresponder a errores de predicción.
 - Considere la proximidad y la dispersión de los clústeres en ambas representaciones, y extraiga conclusiones sobre la capacidad del modelo para separar las clases de imágenes en las dos capas de la red.

Los gráficos t-SNE obtenidos son:

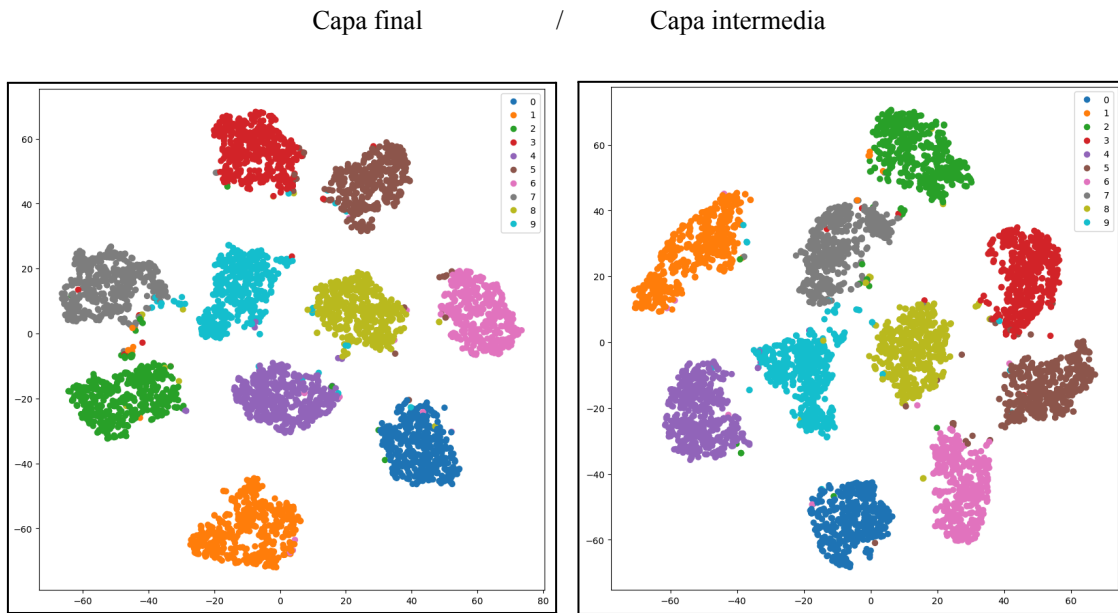


Figura 3: proyecciones t-SNE para las capas intermedia y final.

Para la construcción de gráficos t-SNE, se toman muestras iterativamente y se van colocando en un plano 2D de acuerdo con sus vecindades en el espacio original. Por ejemplo, para la capa intermedia primera de la red, definida en $R^{28 \times 28 \times 32}$, se irían tomando imágenes de *train* (muestras, puntos en ese espacio de altas dimensiones) y colocándolas en un espacio R^2 conservando en la medida de lo posible las distancias originales en el espacio $R^{28 \times 28 \times 32}$. El resultado de esto es bastante parecido al de PCA, pues se obtienen clústers con un cierto significado semántico: representan dígitos concretos.

Si aceptamos la teoría de que efectivamente cada clúster representa un dígito, los errores de predicción corresponderían a aquellas muestras (puntos en el espacio ahora de dos dimensiones) que no se encuentran en el clúster que deberían. En concreto, para la capa intermedia, estos puntos serían aquellos coloreados de un color distinto al del clúster al que pertenecen. Por ejemplo, en el clúster azul oscuro asociado al dígito cero en la representación de la derecha, hay observaciones rosas, verdes y marrones, correspondientes a los dígitos seis, dos y cinco. Estas observaciones supondrían errores de predicción, aunque la capa intermedia no es la que hace las predicciones.

Basándonos en la proximidad y la dispersión de los clústers en ambas representaciones, podemos concluir que el modelo es capaz de separar las imágenes según su etiqueta en ambos casos. Es claro que la dispersión es, en cierta medida, menor en la capa final que en la capa intermedia (los clústers parecen seguir estructuras más concentradas alrededor de su núcleo). Esto es algo esperable, pues la red ha seguido entrenando. Sin embargo, este efecto es menor de lo que cabría esperar (personalmente esperábamos una mejora considerable entre una capa intermedia y la final). Por otra parte, la distancia entre clústers es, a ojo, bastante parecida en ambas gráficas (esto implica que los núcleos sí se están prediciendo correctamente en ambas representaciones, algo normal teniendo en cuenta que representa un promedio). Este efecto se podría observar con mayor facilidad si se representara el centro de cada clúster.

- Dadas las diferencias entre las representaciones t-SNE de ambas capas y la arquitectura de la red implementada: 1) identifique qué capas de la red corresponden a cada representación extraída (intermedia y final), y 2) proponga una forma de reducir el tamaño del modelo con una penalización baja en la precisión de clasificación.

1. Representaciones extraídas para t-SNE.

- 1.1. La capa intermedia de la que hablamos es la capa de justo después del reshape para el aplanado, según se puede observar en la definición del método *forward*. Es por tanto una capa densa, no una convolucional. Esta capa está definida en $R^{14 \times 14 \times 32} \rightarrow R^{128}$, y como las features son extraídas justo después de esta capa, la redimensión de la que se hablaba antes va de $R^{128} \rightarrow R^2$. Por su parte, la capa final sólo puede corresponder a la capa densa mediante la cual se obtienen los outputs.
- 1.2. La capa final es la capa de resultados antes de ser interpretados como probabilidad usando la función *softmax*. Como se puede apreciar en la arquitectura, la salida es de dimensión *output_dim*, que en nuestro caso, para resolver el problema de clasificación de dígitos de MNIST es de dimensión 10. Y por tanto la proyección que usa t-SNE es $R^{10} \rightarrow R^2$.

Las features extraídas de la imagen hacen referencia al estado de los datos después de pasar por la capa intermedia y la final respectivamente.

2. Reducción del tamaño del modelo

Para reducir el tamaño del modelo lo suyo es analizar primero cómo se reparten los parámetros entre sus capas. Podemos observar que casi todos se concentran en la primera capa totalmente conectada (fc1), que recibe $14 \times 14 \times 32$ valores y los transforma en 128, lo que supone unos 803.000 parámetros. Las dos capas convolucionales, en cambio, suman menos de 10.000. Por este motivo, no resulta útil sustituir los kernels por otros más pequeños o factorizados (por ejemplo, un kernel de 3×3 por dos de 3×1 y 1×3), ya que la red solo utiliza kernels de 3×3 y el ahorro sería de unos 3.000 parámetros, apenas perceptible frente al total. Lo más adecuado es actuar sobre la entrada de fc1. Para ello se añadiría una segunda capa de max pooling de 2×2 con stride 2 antes de aplanar, de modo que los mapas de características pasarían de 14×14 a 7×7 y la entrada de fc1 se reduciría de $14 \times 14 \times 32$ a $7 \times 7 \times 32$ valores. Con este cambio fc1 baja de unos 803.000 a unos 201.000 parámetros, y el modelo completo se reduce aproximadamente un 74 %. Como el pooling solo elimina parte de la información espacial y las capas convolucionales no se modifican, la pérdida de precisión esperada es pequeña, algo que habría que comprobar comparando ambos modelos sobre el conjunto de validación.

1.2 AlexNet

- Número de parámetros del modelo AlexNet

	Nº parámetros entrenables
AlexNet	6,199,498

- Incluya las curvas de entrenamiento y validación para 15 épocas. Indique también la mejor precisión obtenida, y en qué época se logra este resultado

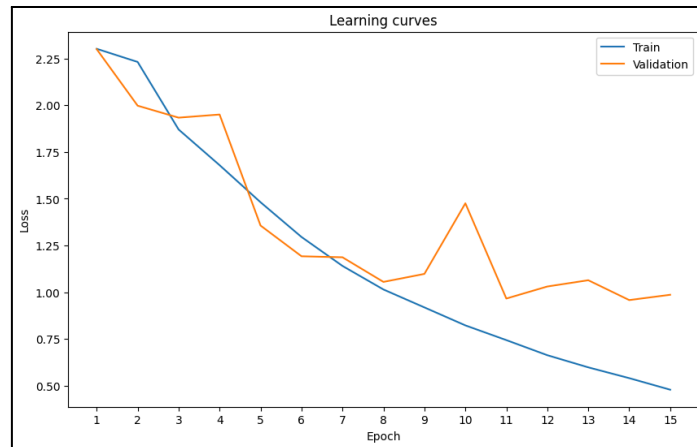


Figura 4: curvas de entrenamiento para la versión de AlexNet de la práctica.

	Mejor precisión (validación)	Época con mejor precisión
AlexNet	70.67%	15

- Comentar las conclusiones sobre la evolución de la *loss* de entrenamiento y validación, y comentar lo que posiblemente está sucediendo después de la época 10. Indique si considera que continuar con más épocas de entrenamiento mejoraría el rendimiento del modelo

En este caso la *loss* de entrenamiento sí está, en promedio, por debajo de la *loss* en validación. Tiene sentido teniendo en cuenta que el número de parámetros después del *dropout* sigue siendo mucho mayor que con la CNN del ejercicio anterior, por lo que las decisiones cambian en mucha menor medida incluso al desactivar la mitad de las neuronas de ciertas capas. Por otra parte, a partir de la época 10 se ve claramente como el modelo comienza a sobreajustar, por lo que las *loss* para entrenamiento sigue bajando mientras que la de validación se mantiene e incluso tiene un pico de retroceso. Continuar con más épocas no tendría demasiado sentido en este contexto, pues se tardaría mucho tiempo en llegar a un pico de interpolación debido a un *double descent* de tipo *epoch wise*. Seguir entrenando el modelo sería por tanto contraproducente; provocaría un mayor sobreajuste.

- Incluya los resultados de precisión por clase (dispone de un bloque de código vacío en el notebook preparado para este propósito).

El modelo brinda la siguiente salida para la precisión del modelo para cada clase:

```
airplane: 73.70% automobile: 79.20% bird: 63.10% cat: 46.60% deer:
72.40% dog: 71.10% frog: 66.30% horse: 79.40% ship: 80.60% truck:
74.30%
```

- Incluya la matriz de confusión. Comente sus resultados atendiendo a las características visuales de las clases de imágenes. No se limite a indicar los errores de clasificación más frecuentes. En su análisis:
 - Comente la posible existencia de grupos de categorías, es decir, conjuntos de varias categorías que se confundan habitualmente entre sí.
 - Describa las características visuales comunes de esos grupos de categorías.
 - A partir de los resultados de la matriz de confusión, formule una hipótesis que explique la distribución de la precisión por clase.

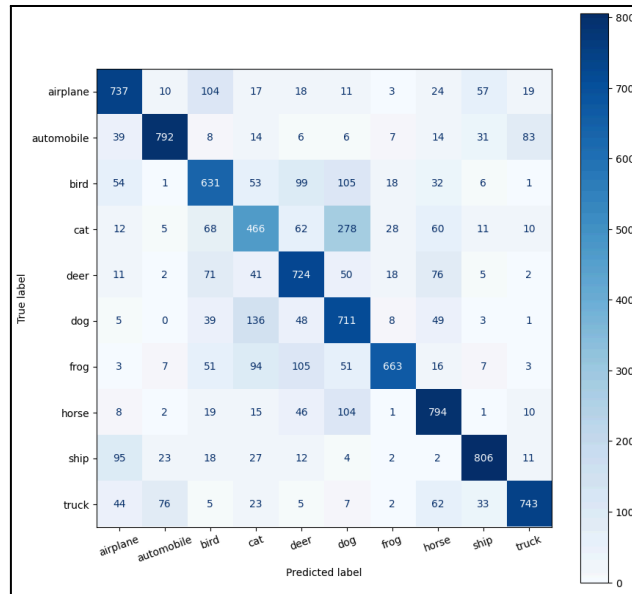


Figura 5: matriz de confusión para las predicciones de la versión de AlexNet de la práctica.

La matriz de confusión muestra dos núcleos. En los vehículos, el modelo confunde sobre todo camiones con coches (76 casos), aviones con camiones (44), barcos con aviones (95) y coches con aviones, algo comprensible a 32×32 píxeles y esperable en una CNN de 2012. Más graves, aunque también esperables, son los cruces entre vehículos y animales, como caballos predichos como camiones (62), pájaros como aviones (54) y aviones como pájaros (104). Los animales concentran el grueso de los errores. Los más frecuentes son gatos predichos como perros (278, más de un tercio) y perros como gatos (136). Ante imágenes parecidas, el modelo tiende a predecir perro, incluso en pájaros (105) y caballos (104): lo esperable serían unas 1.000 predicciones de perro en un conjunto balanceado como CIFAR-10 y hay 1.327.

Nuestra hipótesis es que el modelo se apoya en pistas globales (color, fondo, silueta) y capta mal el detalle fino. AlexNet está pensada para imágenes mucho mayores, y su gran kernel y su pooling reducen demasiado rápido la resolución de CIFAR-10. Además, su capacidad se concentra en las capas totalmente conectadas, lo que favorece memorizar las clases variables pese al dropout, y desde la época 10 ya hay sobreajuste. Por eso los objetos rígidos (barco, coche, camión y avión, con un 74-81 %) y el caballo (79 %, por su silueta) se clasifican mejor, mientras que el gato es la peor clase (47 %): es muy variable y lo que lo separa del perro (orejas, hocico) es el detalle que se pierde. El perro, también variable, absorbe los casos ambiguos, incluso de pájaros o caballos.

Como contraste, compararíamos la precisión por clase en train y test: si el gato falla también en train, la causa sería la arquitectura; si solo falla en test, el sobreajuste. También mediríamos la dispersión de las features por clase en t-SNE, que debería ser mayor en gato y perro.

- Incluya los resultados t-SNE para la capa última capa de la red: analice estos resultados (proximidad, dispersión, agrupación de clústeres) teniendo en cuenta la apariencia de las imágenes de las diferentes clases, sus características típicas y compare los resultados con los resultados t-SNE en el dataset MNIST.

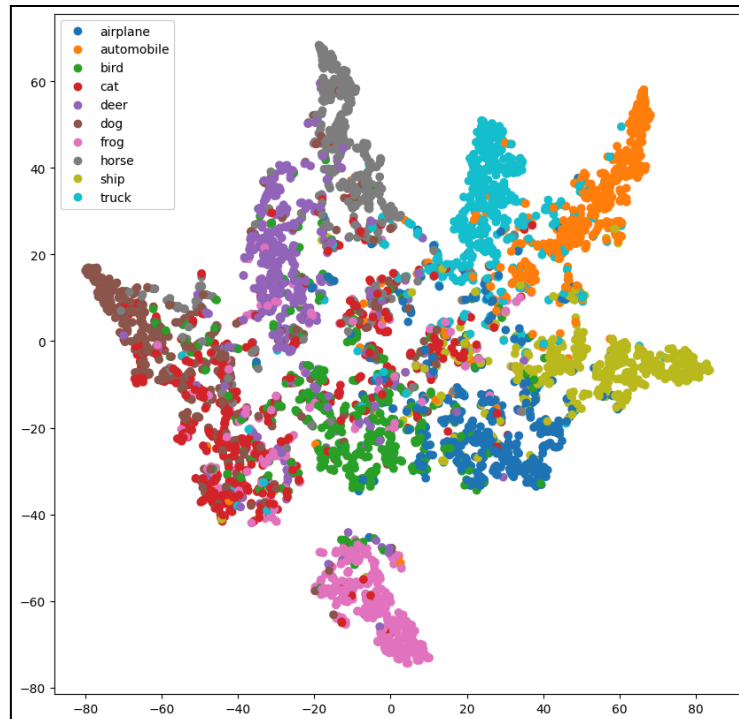


Figura 6: proyección t-SNE de la última capa de la versión de AlexNet para la práctica.

Esta figura puede confirmar la hipótesis que planteamos anteriormente. La dispersión de las clases perro y gato es bastante alta y además están solapadas en gran medida. Por otra parte, es claro como una gran parte de los puntos lejos de su “clúster” (estos clústeres, si es que se les puede llamar así, están claramente peor definidos que para MNIST, como era de esperar) caen cerca de la nube de puntos de perro, absorbiendo esta clase todos los casos ambiguos (de ahí que el modelo tenga sesgo a perro, tal vez porque es el estereotipo de animal; el animal del que partes para construir el resto de animales).

Otros aspectos que refuerzan lo analizado en la matriz de confusión: existen agrupaciones para los vehículos (nubes de puntos a la derecha de la figura), y para los animales (agrupados a su izquierda). También existe solape entre los aviones y los pájaros (algo razonable) y el clúster de las ranas está muy separado del resto (algo que también tiene sentido por los tonos cromáticos y las formas extravagantes).

1.3 Red residual (ResNet) básica

- Incluya en la tabla el número de parámetros del modelo BasicResNet implementado

	# parámetros entrenables
BasicResNet	163,210

- Incluya las curvas de entrenamiento y validación para 15 épocas. Indique también el mejor resultado de precisión y en qué época se logra

	Mejor precisión (validación)	Época con la mejor precisión
BasicResNet	74.47%	14

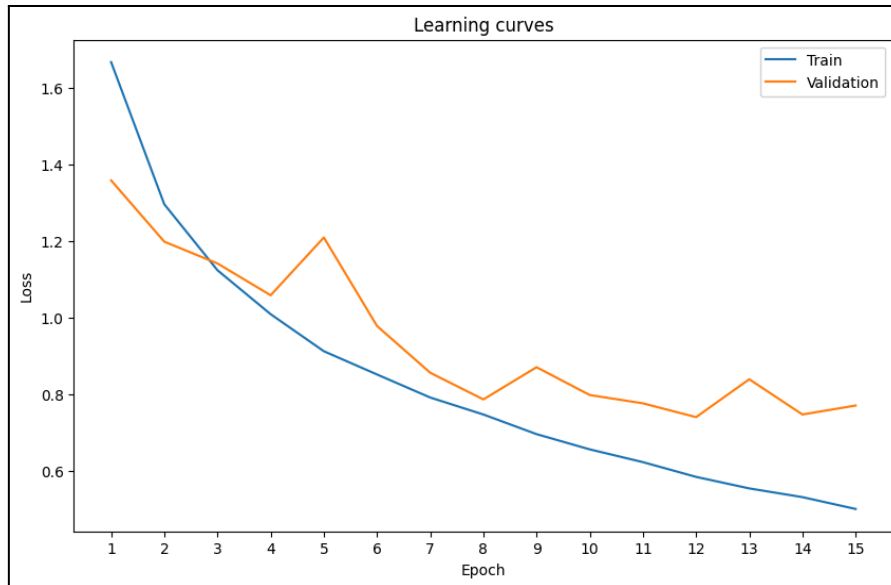


Figura 7: curvas de entrenamiento de la versión de la ResNet de la práctica para los conjuntos de train y validación.

Las curvas de aprendizaje muestran un descenso sostenido de la pérdida de entrenamiento, que pasa de 1.67 a cerca de 0.5 tras 15 épocas. En las primeras épocas, la pérdida de entrenamiento resulta superior a la de validación, algo esperable por dos motivos: se calcula como la media de todos los lotes de la época, incluidos los iniciales, cuando el modelo aún apenas había aprendido, y además el *dropout* solo se aplica durante el entrenamiento, mientras que la validación se evalúa con la red completa y los pesos ya actualizados. A partir de la época 3 las curvas se cruzan y la pérdida de validación se estabiliza en torno a 0.75-0.8, con picos puntuales (épocas 5, 9 y 13). La brecha creciente entre ambas curvas apunta a un sobreajuste leve, aunque la validación no llega a empeorar de forma sostenida.

- Comente sobre la relación entre la complejidad computacional del modelo (tamaño del modelo) y la precisión obtenida para los modelos AlexNet y ResNet implementados. Compare esta relación en ambas arquitecturas para llegar a una conclusión comparativa sobre la eficiencia de ambas.

La relación tamaño-precisión ya ha sido mostrada en este documento, pero conviene resumirla en esta tabla:

	AlexNet	ResNet
Parámetros	~6M	~160k
Accuracy	70.67%	74.47%

Tabla 1: relación tamaño-precisión de las implementaciones de AlexNet y ResNet de la práctica.

Resumidamente, es fácil ver que la ResNet tiene unas 38 veces menos parámetros y aun así consigue ~3.8 puntos porcentuales más de accuracy. En este caso, más tamaño no se traduce en mejor rendimiento. Las capas convolucionales de AlexNet suman ~0.9M parámetros, mientras que sus capas lineales (512→2048, 2048→2048, 2048→clases) aportan más de 5 M, es decir, casi un 85 % del modelo. Estas capas, al no tener estructura espacial ni compartición de pesos, son más propensas al sobreajuste. En realidad, supone una carga para el modelo, pues son capas “caras” pero poco rentables (y que, de sobreajustar, estarían empeorando la calidad de las predicciones del modelo).

Por su parte, la ResNet presenta casi todas las ventajas: casi todos sus parámetros están en convoluciones, sus conexiones residuales facilitan el flujo del gradiente por las capas, la normalización del batch estabiliza, aporta eficacia y regulariza y además es una red más profunda, que permite construir jerarquías de características más ricas con menos pesos. Solo quedaría comprobar esto en términos de eficiencia computacional (recordemos que AlexNet, si algo bueno tiene, es precisamente que puede ser ejecutada en paralelo). Pero parece que con la figura presentada en clases de teoría es suficiente como para ver que ResNet es, en términos de eficiencia, muy parecida a AlexNet, por lo que en este trade-off sale ganando:

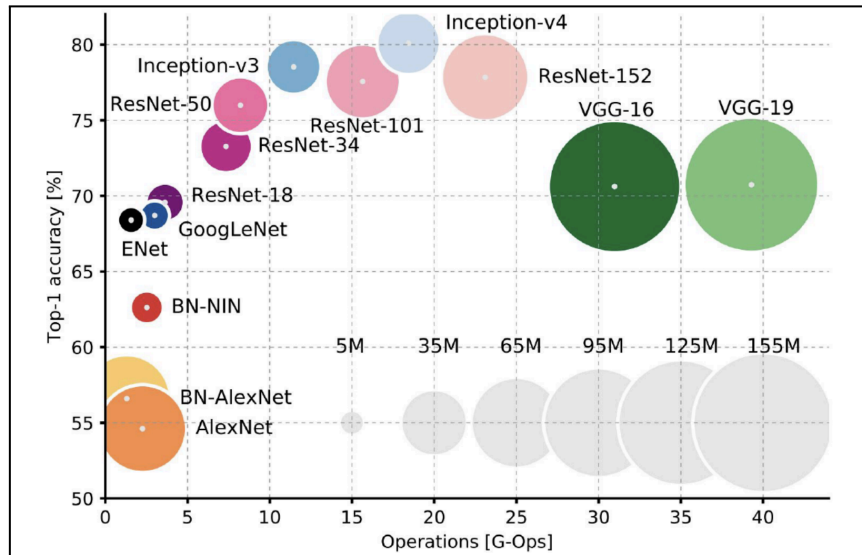


Figura 8: Canziani et al. An Analysis of Deep Neural Network Models for Practical Applications. arXiv 2016, obtenida de las diapositivas de las clases de teoría.